

```
import logging
import json
import os
from datetime import datetime
from telegram import (
    Update,
    InlineKeyboardButton,
    InlineKeyboardMarkup,
)
from telegram.ext import (
    Application,
    CommandHandler,
    CallbackQueryHandler,
    MessageHandler,
    filters,
    ContextTypes,
)

# ——— تنظیمات


---



BOT_TOKEN = os.environ.get("BOT_TOKEN")
ADMIN_ID = 5570181068
ADMIN_USERNAME = "@pouya_20230"
CARD_NUMBER = "6104338654792650"
CARD HOLDER = "پیروزمند"
DB_FILE = "database.json"

# ——— قیمت‌ها


---


```

```
SERVICES = {
    "follower": {"name": "👥 فالوور",
    "price_per_1000": 150000, "unit": "فالوور"},
    "like": {"name": "❤️ لایک",
    "price_per_1000": 60000, "unit": "لایک"},
    "post_view": {"name": "👁️ ویو پست",
    "price_per_1000": 30000, "unit": "ویو"},
    "comment": {"name": "💬 کامنت",
    "price_per_1000": 1400000, "unit": "کامنت"},
    "story_view": {"name": "📖 ویو استوری",
    "price_per_1000": 30000, "unit": "ویو"},
    "save": {"name": "📌 سیو",
    "price_per_1000": 30000,
    "unit": "سیو"},
    "share": {"name": "➦ شیر پست",
    "price_per_1000": 30000, "unit": "شیر"},
}

EXPLORE_PACKAGES = {
    "bronze": {"name": "🥉 بسته برنزی",
    "price": 200000, "desc": "افزایش اولیه تعامل برای ورود به اکسپلور",
    "unit": "ورود به اکسپلور"},
    "silver": {"name": "🥈 بسته",
    "price": 400000, "desc": "تعامل بیشتر +",
    "unit": "لایک و سیو تقویت شده"},
    "gold": {"name": "🥇 بسته طلایی",
    "price": 550000, "desc": "بسته جامع با فالوور، لایک، سیو و ویو"},
    "diamond": {"name": "💎 بسته الماسی",
    "price": 750000, "desc": "کاملترین بسته با بیشترین شانس اکسپلور"},
}
```

```

DISCOUNT_CODES = {
    "WELCOME10": 10,
    "VIP20": 20,
}

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# ————— دیتابیس —————


---



def load_db():
    if os.path.exists(DB_FILE):
        with open(DB_FILE, "r",
            encoding="utf-8") as f:
            return json.load(f)
    return {"users": {}, "orders": []}

def save_db(db):
    with open(DB_FILE, "w", encoding="utf-8") as f:
        json.dump(db, f,
            ensure_ascii=False, indent=2)

def get_user(db, user_id):
    uid = str(user_id)
    if uid not in db["users"]:
        db["users"][uid] = {"wallet": 0,
            "orders": [], "pending_charge": 0}
    return db["users"][uid]

def format_price(n):
    return f"{n:,.} تومان"

```

————— کیبوردها

```
def main_keyboard():
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("🛒 سفارش  
خدمات", callback_data="services")],
        [InlineKeyboardButton("🛒 سبد خرید  
اکسپلور", callback_data="explore")],
        [InlineKeyboardButton("💰 کیف پول",  
callback_data="wallet"),
         InlineKeyboardButton("📄 سفارشات  
من", callback_data="my_orders")],
        [InlineKeyboardButton("🎁 کد تخفیف",  
callback_data="discount")],
        [InlineKeyboardButton("☎ پشتیبانی",  
url=f"https://t.me/{ADMIN_USERNAME.replace(
            '@', '')}")]
    ])

def services_keyboard():
    buttons = []
    for key, svc in SERVICES.items():
        price =
        format_price(svc["price_per_1000"])

        buttons.append([InlineKeyboardButton(
            f"{svc['name']} – {price} /  
هزار",
            callback_data=f"order_{key}"
        )])

    buttons.append([InlineKeyboardButton("⬅️  
بازگشت", callback_data="main")])
```

```

        return InlineKeyboardMarkup(buttons)

def explore_keyboard():
    buttons = []
    for key, pkg in
EXPLORE_PACKAGES.items():

    buttons.append([InlineKeyboardButton(
        f"{pkg['name']} –
{format_price(pkg['price'])}",
        callback_data=f"explore_{key}"
    )])

    buttons.append([InlineKeyboardButton("⬅️ BACK
بازگشت", callback_data="main")])
    return InlineKeyboardMarkup(buttons)

def wallet_keyboard():
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("➕ افزایش
موجودی", callback_data="add_balance")],
        [InlineKeyboardButton("⬅️ BACK
بازگشت", callback_data="main")],
    ])

def back_keyboard(target="main"):
    return
InlineKeyboardMarkup([[InlineKeyboardButton
("⬅️ BACK
بازگشت", callback_data=target)])

# ————— استارت —————

async def start(update: Update, ctx:

```

```

ContextTypes.DEFAULT_TYPE):
    db = load_db()
    user = update.effective_user
    get_user(db, user.id)
    save_db(db)
    text = (
        f"👋 عزیز {user.first_name} سلام\n\n"
        "به ربات خدمات اینستاگرام خوش اومدی 🌟\n\n"
        "از منوی زیر خدمات مورد نظرت رو انتخاب کن:"
    )
    if update.message:
        await
        update.message.reply_text(text,
            reply_markup=main_keyboard())
    else:
        await
        update.callback_query.edit_message_text(text,
            reply_markup=main_keyboard())

# ————— کالکها —————

async def callback_handler(update: Update,
    ctx: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    data = q.data

    if data == "main":
        await start(update, ctx)

    elif data == "services":
        await q.edit_message_text(
            "یک سرویس رو\n\nخدمات اینستاگرام 📦"

```

```

انتخاب کن",

reply_markup=services_keyboard()
)

elif data == "explore":
    text = (
        "🚀 بسته‌های ورود به اکسپلور\n\n"
        "با خرید این بسته‌ها، خدمات ترکیبی روی "
        "پست شما انجام می‌شه "
        "تا شانس ورود به صفحه اکسپلور اینستاگرام"
        "\n\nافزایش پیدا کنه
    )
    for pkg in
EXPLORE_PACKAGES.values():
        text += f"{pkg['name']} -
{format_price(pkg['price'])}\n
{pkg['desc']}\n\n"
        await q.edit_message_text(text,
reply_markup=explore_keyboard())

elif data.startswith("explore_"):
    key = data.replace("explore_", "")
    pkg = EXPLORE_PACKAGES[key]
    ctx.user_data["pending_order"] = {
        "type": "explore",
        "key": key,
        "name": pkg["name"],
        "price": pkg["price"],
        "quantity": 1,
    }
    await q.edit_message_text(
        f"{pkg['name']}\n\n"
        f"💰 قیمت:

```

```

{format_price(pkg['price'])}\n"
    f"📝 {pkg['desc']}\n\n"
    "لطفاً لینک پست اینستاگرامت رو ارسال
کن:",

reply_markup=back_keyboard("explore")
)
ctx.user_data["step"] =
"awaiting_link"

elif data.startswith("order_"):
    key = data.replace("order_", "")
    svc = SERVICES[key]
    ctx.user_data["pending_order"] = {
        "type": "service",
        "key": key,
        "name": svc["name"],
        "price_per_1000":
svc["price_per_1000"],
        "unit": svc["unit"],
    }
    await q.edit_message_text(
        f"{svc['name']}\n\n"
        f"💰 قیمت:
{format_price(svc['price_per_1000'])} به ازای
۱۰۰۰ {svc['unit']}\n\n"
        "لطفاً لینک پست یا پروفایل اینستاگرامت رو
ارسال کن",

reply_markup=back_keyboard("services")
)
ctx.user_data["step"] =
"awaiting_link"

```



```

elif data == "wallet":
    db = load_db()
    user = get_user(db, q.from_user.id)
    await q.edit_message_text(
        f"💰 کیف پول شما \n\n"
        f"موجودی فعلی: {format_price(user['wallet'])}\n\n"
        f"برای افزایش موجودی، دکمه زیر رو بزن"
        f"👉",
        reply_markup=wallet_keyboard()
    )

elif data == "add_balance":
    await q.edit_message_text(
        f"🇮🇷 مبلغ مورد نظر (به افزایش موجودی \n\n"
        f":تومان) رو وارد کن"

    reply_markup=back_keyboard("wallet")
    )
    ctx.user_data["step"] =
    "awaiting_charge_amount"

elif data == "my_orders":
    db = load_db()
    user = get_user(db, q.from_user.id)
    orders = user.get("orders", [])
    if not orders:
        text = "📋 هنوز سفارشات من \n\n"
        text = "سفارشی ثبت نکردی"
    else:
        text = "📋 سفارشات من \n\n"
        for i, o in
        enumerate(reversed(orders[-10:]), 1):
            status_emoji = "✅" if

```

```

o["status"] == "completed" else "⌚" if
o["status"] == "pending" else "❌"
        text += (
            f"{i}. {o['name']}\n"
            f"    📌\n"
            f"{format_price(o['price'])}\n"
            f"    🔗 {o.get('link',
            '-')}\n"
            f"    📅 {o['date']}\n"
            f"    {status_emoji}"
            وضعيت: {o['status']}\n\n"
        )
        await q.edit_message_text(text,
reply_markup=back_keyboard())

    elif data == "discount":
        await q.edit_message_text(
            "🎁 کد تخفیف خود را وارد کنید\n\nکد تخفیف:",
            reply_markup=back_keyboard()
        )
        ctx.user_data["step"] =
"awaiting_discount"

    elif data.startswith("confirm_order_"):
        await confirm_order(update, ctx)

    elif data == "cancel_order":
        ctx.user_data.clear()
        await start(update, ctx)

async def confirm_order(update: Update,
ctx: ContextTypes.DEFAULT_TYPE):

```

```

q = update.callback_query
db = load_db()
user_data = get_user(db,
q.from_user.id)
order =
ctx.user_data.get("pending_order", {})
price = order["price"]
discount =
ctx.user_data.get("discount_percent", 0)
final_price = int(price * (1 - discount
/ 100))

if user_data["wallet"] < final_price:
    shortage = final_price -
user_data["wallet"]
    await q.edit_message_text(
        f"❌ موجودی کافی نیست \n\n"
        f"موجودی شما:
{format_price(user_data['wallet'])}\n"
        f"مبلغ سفارش:
{format_price(final_price)}\n"
        f"کمبود:
{format_price(shortage)}\n\n"
        ". لطفاً ابتدا کیف پول خود را شارژ کنید",

reply_markup=InlineKeyboardMarkup([
    [InlineKeyboardButton("💰",
callback_data="add_balance")],
    [InlineKeyboardButton("⬅️",
callback_data="main")],
    ])
    )
    return

```

```

user_data["wallet"] -= final_price
new_order = {
    "id": len(db["orders"]) + 1,
    "name": order["name"],
    "price": final_price,
    "link": order.get("link", "-"),
    "quantity": order.get("quantity",
1),
    "date":
datetime.now().strftime("%Y-%m-%d %H:%M"),
    "status": "pending",
    "user_id": str(q.from_user.id),
}
db["orders"].append(new_order)
user_data["orders"].append(new_order)
save_db(db)
ctx.user_data.clear()

discount_text = f"\n🎁 تخفیف:
{discount}%" if discount else ""
await q.edit_message_text(
    f"✅ سفارش با موفقیت ثبت شد\n\n"
    f"📦 {new_order['name']}\n"
    f"🔗 {new_order['link']}\n"
    f"💰 مبلغ پرداخت شده:
{format_price(final_price)}
{discount_text}\n"
    f"📅 تاریخ: {new_order['date']}\n"
    f"🆔 کد سفارش: #
{new_order['id']}\n\n"
    "⌚ سفارش شما در حال پردازش است",
    reply_markup=back_keyboard()
)

```

```

# اطلاع‌رسانی به ادمین
try:
    await ctx.bot.send_message(
        ADMIN_ID,
        f"🛒 سفارش جدید\n\n"
        f"کاربر: {q.from_user.full_name}"
        f"(ID: {q.from_user.id})\n"
        f"📦 {new_order['name']}\n"
        f"🔗 {new_order['link']}\n"
        f"💰 {format_price(final_price)}\n"
        f"# کد سفارش {new_order['id']}"
    )
except Exception:
    pass

```

————— پیام‌های متنی

```

async def message_handler(update: Update,
                           ctx: ContextTypes.DEFAULT_TYPE):
    step = ctx.user_data.get("step")
    text = update.message.text.strip() if
update.message.text else ""

    if step == "awaiting_link":
        ctx.user_data["pending_order"]
["link"] = text
        order =
ctx.user_data["pending_order"]

        if order["type"] == "explore":

```

```

        ctx.user_data["step"] =
"ready_to_confirm"
        price = order["price"]
        discount =
ctx.user_data.get("discount_percent", 0)
        final_price = int(price * (1 -
discount / 100))
        discount_text = f"\n🎁 تخفیف:
{discount}% ← {format_price(final_price)}"
if discount else ""
        await
update.message.reply_text(
        f"✅ تأیید سفارش\n\n"
        f"📦 {order['name']}\n"
        f"🔗 {text}\n"
        f"💰 قیمت:
{format_price(price)}{discount_text}\n\n"
        "آیا سفارش را تأیید می‌کنی؟",

reply_markup=InlineKeyboardMarkup([

[InlineKeyboardButton("✅ تأیید و پرداخت",
callback_data="confirm_order_")],

[InlineKeyboardButton("❌ انصراف",
callback_data="cancel_order")],

])
    )
    else:
        ctx.user_data["step"] =
"awaiting_quantity"
        svc = SERVICES[order["key"]]
        await
update.message.reply_text(

```

```

        f"مورد نیاز {svc['unit']} تعداد"
    را وارد کن
    f"(مثلاً ۱۰۰۰ یا ۵۰۰۰)\n\n"
    f"💡 حداقل سفارش: ۱۰۰۰ عدد",

    reply_markup=back_keyboard("services")
    )

    elif step == "awaiting_quantity":
        try:
            qty = int(text.replace(",", ""))
            if qty < 1000:
                await
            update.message.reply_text("❌ حداقل سفارش ۱۰۰۰ عدد است")
            return
            order =
            ctx.user_data["pending_order"]
            price =
            int(order["price_per_1000"] * qty / 1000)
            order["quantity"] = qty
            order["price"] = price
            discount =
            ctx.user_data.get("discount_percent", 0)
            final_price = int(price * (1 -
            discount / 100))
            discount_text = f"\n🎁 تخفیف:
            {discount}% ← {format_price(final_price)}"
            if discount else ""
            await
            update.message.reply_text(
                f"✅ تأیید سفارش\n\n"
                f"📦 {order['name']}\n"

```

```

        f"🔗 {order['link']}\n"
        f"🧾 تعداد: {qty:},{\n"
        {order['unit']}\n"
        f"💰 قیمت:
        {format_price(price)}{discount_text}\n\n"
        "آیا سفارش را تأیید می‌کنی؟",

        reply_markup=InlineKeyboardMarkup([

            [InlineKeyboardButton("✅ تأیید و پرداخت",
                callback_data="confirm_order_")],

            [InlineKeyboardButton("❌ انصراف",
                callback_data="cancel_order")],

        ])

    )
    ctx.user_data["step"] =
    "ready_to_confirm"
    except ValueError:
        await
        update.message.reply_text("❌ لطفاً یک عدد معتبر
        وارد کن")

    elif step == "awaiting_charge_amount":
        try:
            amount = int(text.replace(",",
            "").replace(".", ""))
            if amount < 10000:
                await
                update.message.reply_text("❌ حداقل مبلغ شارژ
                ۱۰,۰۰۰! تومان است")
            return
            ctx.user_data["pending_charge"]
            = amount

```



```

        ctx.user_data["step"] =
"awaiting_receipt"
        await
update.message.reply_text(
    f"🇮🇷 اطلاعات واریز\n\n"
    f"مبلغ:
{format_price(amount)}\n\n"
    f"شماره
کارت:\n`{CARD_NUMBER}`\n"
    f"به نام: {CARD_HOLDER}\n\n"
    f"پس از واریز، شماره تراکنش یا تصویر"
    f":فیش را اینجا ارسال کن",
    parse_mode="Markdown",

reply_markup=back_keyboard("wallet")
)
except ValueError:
    await
update.message.reply_text("❌ لطفاً مبلغ را به
صورت عدد وارد کن")

elif step == "awaiting_receipt":
    amount =
ctx.user_data.get("pending_charge", 0)
    db = load_db()
    user = get_user(db,
update.effective_user.id)
    user["pending_charge"] = amount
    save_db(db)
    ctx.user_data.clear()

    receipt_text = text if not
update.message.photo else "📷 تصویر فیش"
    await update.message.reply_text(

```

```

        f"✅ درخواست شارژ ثبت شد \n\n"
        f"مبلغ:
{format_price(amount)}\n"
        f"فیش: {receipt_text}\n\n"
        f"پس از تأیید توسط ادمین، موجودی شما"
        f"افزایش خواهد یافت \n"
        f"معمولاً کمتر از ۳۰ دقیقه طول می‌کشد"
        f"⏳",
        reply_markup=main_keyboard()
    )

    try:
        await ctx.bot.send_message(
            ADMIN_ID,
            f"$👉 درخواست شارژ جدید \n\n"
            f"کاربر:
{update.effective_user.full_name} (ID:
{update.effective_user.id})\n"
            f"مبلغ:
{format_price(amount)}\n"
            f"فیش: {receipt_text}\n\n"
            f"برای
تأیید: \n/approve_{update.effective_user.id}_{
amount}"
        )
    except Exception:
        pass

    elif step == "awaiting_discount":
        code = text.upper()
        if code in DISCOUNT_CODES:
            discount = DISCOUNT_CODES[code]

    ctx.user_data["discount_percent"] =

```

```

discount
    ctx.user_data["step"] = None
    await
update.message.reply_text(
    f"✅ کد تخفیف اعمال شد!\n\n"
    f"🎁 {discount}% تخفیف برای
سفارش بعدی شما فعال است",

    reply_markup=main_keyboard()
)
else:
    await
update.message.reply_text(
    "❌ کد تخفیف نامعتبر است",

    reply_markup=back_keyboard()
)

# ————— دستورات ادمین —————
—————

async def approve_charge(update: Update,
ctx: ContextTypes.DEFAULT_TYPE):
    if update.effective_user.id !=
ADMIN_ID:
        return
    try:
        parts =
update.message.text.split("_")
        target_id = int(parts[1])
        amount = int(parts[2])
        db = load_db()
        user = get_user(db, target_id)

```

```

        user["wallet"] += amount
        user["pending_charge"] = 0
        save_db(db)
        await
    update.message.reply_text(f"✅  
{format_price(amount)} به کاربر {target_id}  
اضافه شد.")
    await ctx.bot.send_message(
        target_id,
        f"✅ موجودی کیف پول شما شارژ  
شد!\n\n"
        f"مبلغ:  
{format_price(amount)}\n"
        f"موجودی جدید:  
{format_price(user['wallet'])}"
    )
except Exception as e:
    await
    update.message.reply_text(f"خطا: {e}")

# — اجرا

```

```

def main():
    app =
    Application.builder().token(BOT_TOKEN).build()
    app.add_handler(CommandHandler("start",
    start))

    app.add_handler(CommandHandler("approve",
    approve_charge))

```

```
app.add_handler(CallbackQueryHandler(callba  
ck_handler))
```

```
app.add_handler(MessageHandler(filters.TEXT  
| filters.PHOTO, message_handler))  
    print("🏠 ریات در حال اجراست...")  
    app.run_polling()
```

```
if __name__ == "__main__":  
    main()
```